

NHẬP MÔN TRÍ TUỆ NHÂN TẠO

Nguyễn Xuân Việt Đức - 22280012

Bài tập lý thuyết - Tuần 1

Câu 1: Breadth-first search

- b : Hệ số phân nhánh tối đa của cây
- d : Độ sâu của lời giải có chi phí thấp nhất

- Complete:
Hoàn chỉnh, nếu b là 1 số hữu hạn

- Time:
$$1 + b + b^2 + b^3 + \dots + b^d = O(b^d)$$

- Space: (Giữ mọi nốt ở trong bộ nhớ)

$$O(b^d)$$

- Optimal:
Tối ưu, nếu chi phí từ nốt này qua nốt khác bằng nhau

Câu 2: Uniform-cost search

- C^* : Chi phí của lời giải tối ưu
- ϵ : Chi phí bước nhỏ nhất

- Complete:
Hoàn chỉnh, Nếu chi phí bước tối thiểu lớn hơn hoặc bằng ϵ
(Đảm bảo thuật toán luôn tìm ra giải pháp)

- Time: (Phụ thuộc vào số nốt có chi phí nhỏ hơn hoặc bằng chi phí của lời giải tối ưu)

$$O\left(b^{1+\lceil \frac{C^*}{\epsilon} \rceil}\right)$$

- Space:

$$O\left(b^{1+\lceil \frac{C^*}{\epsilon} \rceil}\right)$$

- Optimal:
Tối ưu, nếu các nút được mở rộng theo thứ tự tăng dần

Câu 3: Depth-first search

- m : Độ sâu tối đa

- Complete:
Không hoàn chỉnh, thuật toán thất bại trong không gian có độ sâu vô hạn, không gian có vòng lặp. Hoàn chỉnh trong không gian tìm kiếm hữu hạn
- Time: Trong trường hợp tệ nhất, DFS có thể phải khám phá hết mọi nhánh của cây tìm kiếm có độ sâu tối đa là m , với b là hệ số phân nhánh.

$$O(b^m)$$

- Space: Chỉ cần giữ lại các nút của một nhánh tại bất kỳ thời điểm nào, vì nó khám phá sâu vào cây.

$$O(bm)$$

- Optimal:
Không tối ưu, thuật toán chỉ trú trọng vào việc đi vào sâu mà không quan tâm đến chi phí, nên thuật toán không tối ưu

Câu 4: Greedy best-first search

- Complete:
Không hoàn chỉnh, nếu bị kẹt trong 1 chu trình hoặc trong không gian phức tạp nó sẽ không thể tìm ra cho dù có cách để tìm ra
- Time: trong trường hợp xấu nhất, thuật toán có thể phải duyệt qua tất cả các nút ở độ sâu m

$$O(b^m)$$

- Space: trong trường hợp xấu nhất, thuật toán có thể phải lưu trữ tất cả các nút ở độ sâu m trong bộ nhớ.

$$O(b^m)$$

- Optimal: Không tối ưu, thuật toán có thể tìm ra giải pháp nhanh chóng nhưng chưa chắc là giải pháp tối ưu nhất

Câu 5: A* search

- Giả sử A* trả về một nút đích không tối ưu G_2 .
- G_2 phải là nút có chi phí thấp nhất trong biên. Giả sử chi phí của giải pháp tối ưu là C^*
- Vì G_2 không tối ưu:

$$f(G_2) = g(G_2) + h(G_2) = g(G_2) > C^*$$

- Bây giờ xét một nút biên n trên đường đi tối ưu đến đích G
- Nếu $h(n)$ là chấp nhận được thì:

$$f(n) = g(n) + h(n) \leq C^*$$

- Chúng ta đã chứng minh rằng $f(n) \leq C^* < f(G_2)$, vì vậy G_2 sẽ không được mở rộng trước n . Do đó A^* phải trả về một giải pháp tối ưu.

Chú ý: $h(G_2) = 0$ vì đây là nút đích

Homework 4.4:

Code:

```
import random
import numpy as np
import matplotlib.pyplot as plt

# Define the cost function and solution checker for 8-queens
7 usages new *
def calculate_attacks(queens):
    """Calculate the number of attacking queen pairs."""
    n = len(queens)
    attacks = 0
    for i in range(n):
        for j in range(i+1, n):
            if queens[i] == queens[j] or abs(queens[i] - queens[j]) == abs(i - j):
                attacks += 1
    return attacks

3 usages new *
def generate_random_board():
    """Generate a random configuration for 8-queens."""
    return [random.randint(a=0, b=7) for _ in range(8)]

def hill_climbing_first_choice():
    """First-choice hill climbing for the 8-queens problem."""
    board = generate_random_board()
    while True:
        current_attacks = calculate_attacks(board)
        if current_attacks == 0:
            return board, 0
        neighbors = [list(board) for _ in range(8)]
        for i in range(8):
            neighbors[i][i] = random.randint(a=0, b=7)
        next_board = random.choice(neighbors)
        next_attacks = calculate_attacks(next_board)
        if next_attacks < current_attacks:
            board = next_board
        else:
            return board, current_attacks
```

```

def steepest_ascent_hill_climbing():
    """Steepest ascent hill climbing for the 8-queens problem."""
    board = generate_random_board()
    while True:
        current_attacks = calculate_attacks(board)
        if current_attacks == 0:
            return board, 0
        neighbors = [list(board) for _ in range(64)]
        for i in range(8):
            for j in range(8):
                if board[i] != j:
                    new_board = list(board)
                    new_board[i] = j
                    neighbors.append(new_board)
        next_board = min(neighbors, key=calculate_attacks)
        next_attacks = calculate_attacks(next_board)
        if next_attacks < current_attacks:
            board = next_board
        else:
            return board, current_attacks

def simulated_annealing():
    """Simulated annealing for the 8-queens problem."""
    board = generate_random_board()
    current_attacks = calculate_attacks(board)
    T = 100
    while T > 0.01:
        if current_attacks == 0:
            return board, 0
        new_board = list(board)
        i = random.randint(a=0, b=7)
        new_board[i] = random.randint(a=0, b=7)
        new_attacks = calculate_attacks(new_board)
        if (new_attacks < current_attacks
            or random.uniform(a=0, b=1)
            < np.exp((current_attacks - new_attacks) / T)):
            board = new_board
            current_attacks = new_attacks
        T *= 0.99
    return board, current_attacks

```

```

def random_restart_hill_climbing():
    """Hill climbing with random restart for the 8-queens problem."""
    for _ in range(100):
        board, cost = steepest_ascent_hill_climbing()
        if cost == 0:
            return board, 0
    return None, None

methods = {
    'First-Choice Hill Climbing': hill_climbing_first_choice,
    'Steepest-Ascent Hill Climbing': steepest_ascent_hill_climbing,
    'Simulated Annealing': simulated_annealing,
    'Random Restart Hill Climbing': random_restart_hill_climbing
}

results = {method: {'cost': [], 'solved': 0} for method in methods}

for method_name, method in methods.items():
    for _ in range(100): # 100 test cases
        board, cost = method()
        if cost == 0:
            results[method_name]['solved'] += 1
            results[method_name]['cost'].append(cost)

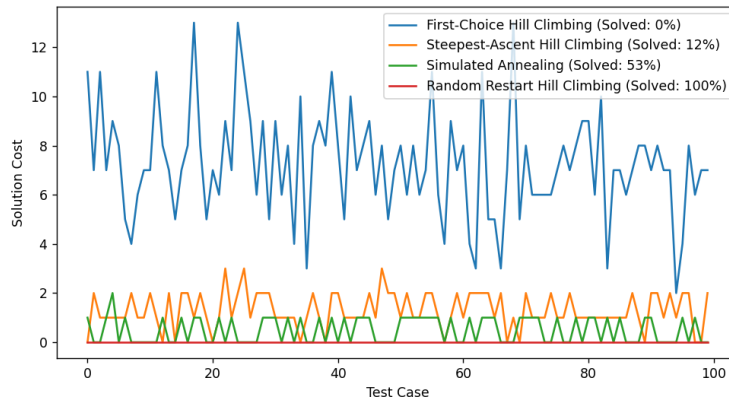
for method_name, data in results.items():
    plt.plot(*args: data['cost'], label=f'{method_name} (Solved: {data["solved"]})%')

plt.xlabel('Test Case')
plt.ylabel('Solution Cost')
plt.legend()
plt.show()

for method_name, data in results.items():
    success_rate = data['solved']
    print(f"{method_name}: {success_rate}% problems solved.")

```

Output:



1. First-Choice Hill Climbing (Leo đồi lựa chọn đầu tiên):

Phương pháp này không giải quyết được bài toán trong bất kỳ trường hợp nào. Nhìn vào biểu đồ, chi phí (bước đi) dao động rất cao và không ổn định, cho thấy phương pháp này dễ bị mắc kẹt trong cực trị cục bộ.

2. Steepest-Ascent Hill Climbing (Leo đồi dốc nhất):

Phương pháp này có tỷ lệ giải quyết bài toán là 12%, cho thấy rằng nó có thể thoát khỏi cực trị cục bộ tốt hơn so với phương pháp lựa chọn đầu tiên. Tuy nhiên, tỷ lệ thành công còn thấp, với một số thử nghiệm yêu cầu nhiều bước hơn.

3. Simulated Annealing (Ủ mô phỏng):

Phương pháp này có tỷ lệ giải quyết bài toán là 53%, cao hơn rõ rệt so với leo đồi. Điều này cho thấy việc sử dụng nhiệt độ trong quá trình tối ưu hóa giúp giảm thiểu khả năng mắc kẹt trong cực trị cục bộ và có khả năng tìm được lời giải hiệu quả hơn.

4. Random Restart Hill Climbing (Leo đồi với khởi động lại ngẫu nhiên):

Phương pháp này đạt tỷ lệ thành công 100%. Điều này cho thấy khởi động lại ngẫu nhiên có hiệu quả cao, vì nó cho phép phương pháp thoát khỏi các cực trị cục bộ thông qua việc bắt đầu lại từ các vị trí ngẫu nhiên mới khi cần thiết.

Câu 6♣ Admissible heuristics

- Khoảng cách Manhattan:

Ô 1: Vị trí hiện tại (3,3) → Vị trí mục tiêu (1,3):

$$h = |3 - 1| + |3 - 3| = 2$$

Ô 2: Vị trí hiện tại (1,2) → Vị trí mục tiêu (1,2):

$$h = |1 - 1| + |2 - 2| = 0$$

Ô 3: Vị trí hiện tại (3,2) → Vị trí mục tiêu (2,3):

$$h = |3 - 2| + |2 - 3| = 2$$

Ô 4: Vị trí hiện tại (1,3) → Vị trí mục tiêu (1,1):

$$h = |1 - 1| + |3 - 1| = 2$$

Ô 5: Vị trí hiện tại (2,1) → Vị trí mục tiêu (2,2):

$$h = |2 - 2| + |1 - 2| = 1$$

Ô 6: Vị trí hiện tại (2,2) → Vị trí mục tiêu (3,1):

$$h = |2 - 3| + |2 - 1| = 2$$

Ô 7: Vị trí hiện tại (1,1) → Vị trí mục tiêu (3,2):

$$h = |1 - 3| + |1 - 2| = 3$$

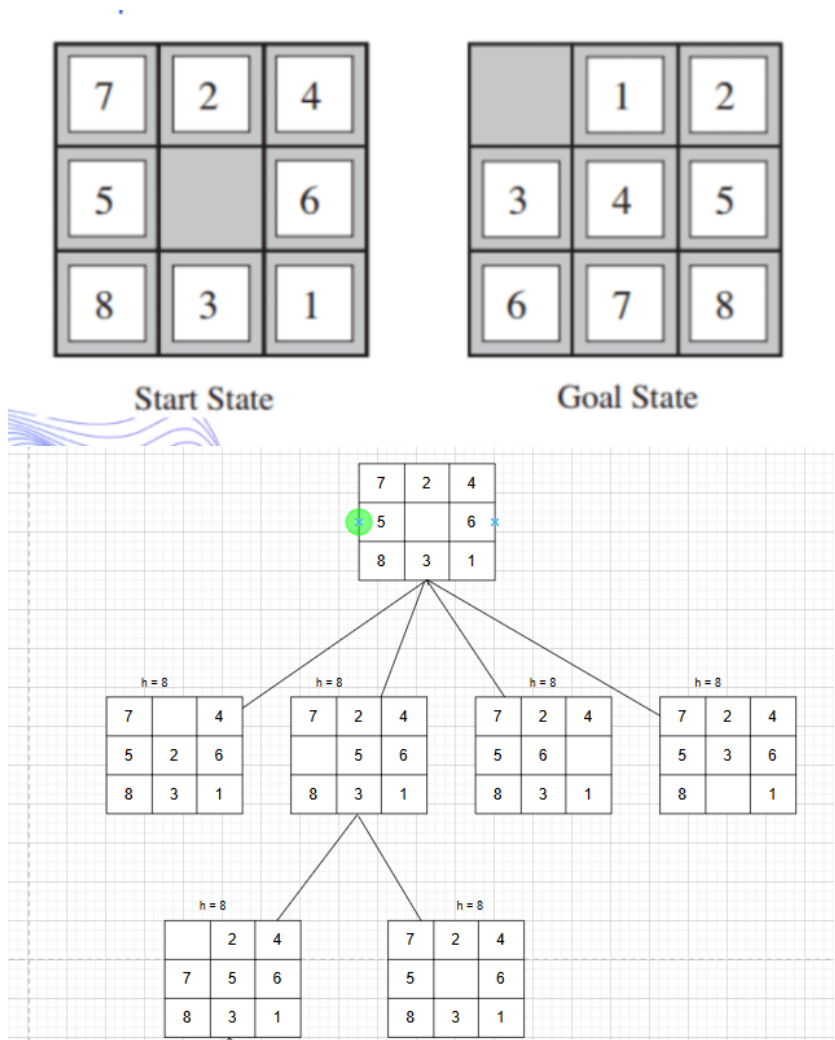
Ô 8: Vị trí hiện tại (3,1) → Vị trí mục tiêu (2,1):

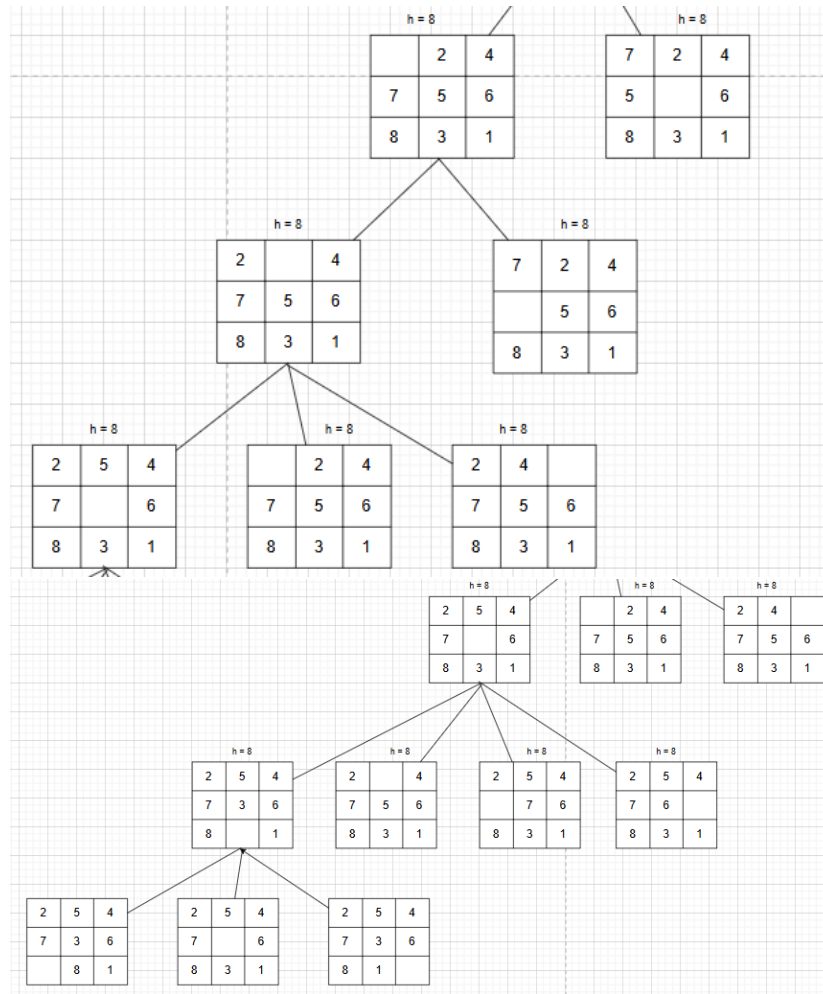
$$h = |3 - 2| + |1 - 1| = 1$$

Tổng hợp h(n):

$$h(n) = 2 + 0 + 2 + 2 + 1 + 2 + 3 + 1 = 13$$

Sử dụng Tree-search để giải The 8 puzzle dưới đây:





Làm tương tự vậy cho đến khi các số nằm ở đúng vị trí như State Goal, (Vì nó quá dài nên em chỉ biểu d

State 1: $\begin{bmatrix} 7 & 2 & 4 \\ 5 & 0 & 6 \\ 8 & 3 & 1 \end{bmatrix}$

State 2: $\begin{bmatrix} 7 & 2 & 4 \\ 0 & 5 & 6 \\ 8 & 3 & 1 \end{bmatrix}$

State 3: $\begin{bmatrix} 0 & 2 & 4 \\ 7 & 5 & 6 \\ 8 & 3 & 1 \end{bmatrix}$

State 4: $\begin{bmatrix} 2 & 0 & 4 \\ 7 & 5 & 6 \\ 8 & 3 & 1 \end{bmatrix}$

State 5:	$\begin{bmatrix} 2 & 5 & 4 \\ 7 & 0 & 6 \\ 8 & 3 & 1 \end{bmatrix}$
State 6:	$\begin{bmatrix} 2 & 5 & 4 \\ 7 & 3 & 6 \\ 8 & 0 & 1 \end{bmatrix}$
State 7:	$\begin{bmatrix} 2 & 5 & 4 \\ 7 & 3 & 6 \\ 0 & 8 & 1 \end{bmatrix}$
State 8:	$\begin{bmatrix} 2 & 5 & 4 \\ 0 & 3 & 6 \\ 7 & 8 & 1 \end{bmatrix}$
State 9:	$\begin{bmatrix} 2 & 5 & 4 \\ 3 & 0 & 6 \\ 7 & 8 & 1 \end{bmatrix}$
State 10:	$\begin{bmatrix} 2 & 5 & 4 \\ 3 & 6 & 0 \\ 7 & 8 & 1 \end{bmatrix}$
State 11:	$\begin{bmatrix} 2 & 5 & 0 \\ 3 & 6 & 4 \\ 7 & 8 & 1 \end{bmatrix}$
State 12:	$\begin{bmatrix} 2 & 0 & 5 \\ 3 & 6 & 4 \\ 7 & 8 & 1 \end{bmatrix}$
State 13:	$\begin{bmatrix} 0 & 2 & 5 \\ 3 & 6 & 4 \\ 7 & 8 & 1 \end{bmatrix}$
State 14:	$\begin{bmatrix} 3 & 2 & 5 \\ 0 & 6 & 4 \\ 7 & 8 & 1 \end{bmatrix}$
State 15:	$\begin{bmatrix} 3 & 2 & 5 \\ 6 & 0 & 4 \\ 7 & 8 & 1 \end{bmatrix}$
State 16:	$\begin{bmatrix} 3 & 2 & 5 \\ 6 & 4 & 0 \\ 7 & 8 & 1 \end{bmatrix}$
State 17:	$\begin{bmatrix} 3 & 2 & 5 \\ 6 & 4 & 1 \\ 7 & 8 & 0 \end{bmatrix}$
State 18:	$\begin{bmatrix} 3 & 2 & 5 \\ 6 & 4 & 1 \\ 7 & 0 & 8 \end{bmatrix}$
State 19:	$\begin{bmatrix} 3 & 2 & 5 \\ 6 & 4 & 1 \\ 0 & 7 & 8 \end{bmatrix}$

$$\begin{array}{lcl}
\text{State 20:} & \begin{bmatrix} 3 & 2 & 5 \\ 0 & 4 & 1 \\ 6 & 7 & 8 \end{bmatrix} \\
\text{State 21:} & \begin{bmatrix} 3 & 2 & 5 \\ 4 & 0 & 1 \\ 6 & 7 & 8 \end{bmatrix} \\
\text{State 22:} & \begin{bmatrix} 3 & 2 & 5 \\ 4 & 1 & 0 \\ 6 & 7 & 8 \end{bmatrix} \\
\text{State 23:} & \begin{bmatrix} 3 & 2 & 0 \\ 4 & 1 & 5 \\ 6 & 7 & 8 \end{bmatrix} \\
\text{State 24:} & \begin{bmatrix} 3 & 0 & 2 \\ 4 & 1 & 5 \\ 6 & 7 & 8 \end{bmatrix} \\
\text{State 25:} & \begin{bmatrix} 3 & 1 & 2 \\ 4 & 0 & 5 \\ 6 & 7 & 8 \end{bmatrix} \\
\text{State 26:} & \begin{bmatrix} 3 & 1 & 2 \\ 0 & 4 & 5 \\ 6 & 7 & 8 \end{bmatrix} \\
\text{State 27:} & \begin{bmatrix} 0 & 1 & 2 \\ 3 & 4 & 5 \\ 6 & 7 & 8 \end{bmatrix}
\end{array}$$